**Predictive Modeling and Regularization Techniques**

# 1. Executive Summary

This report explores the California Housing Prices dataset with the goal of predicting house prices and and using machine learning to group them into different price ranges. The analysis begins with data exploration, where summary statistics and visualizations were used to identify key relationships between variables. Missing data was filled with median, and the outliers are processed by IQR to reduce the influence of outliers on the model. Feature engineering, including transforming categorical data, was used to enhance the models. The multicollinearity between features like *total_rooms*, *total_bedrooms*, and *households* was solved by creating new features, such as *rooms_per_household* and *bedrooms_per_population*, which helped to reduce redundancy. When applying probability theory, a key feature was modeled using a theoretical distribution and evaluated using the Kolmogorov-Smirnov test. Conditional probabilities were calculated to analyze the relationships between features like median income and house prices, with expected value and variance used to understand its behavior.

Linear regression was applied to predict house prices, but residual analysis indicated underfitting, as the model struggled to capture more complex patterns in the data. Polynomial regression was also tested but led to overfitting. For classification, the continuous house price variable was divided into Low, Medium, and High categories, and logistic regression was used to predict these categories. The model's performance was evaluated through metrics such as accuracy, precision, recall, and F1-score, with cross-validation showing good performance for the Low and High categories, but challenges in distinguishing the Medium category.

Regularization techniques, such as Ridge (L2) and Lasso (L1), were applied to the models. However, as the original models showed signs of underfitting, regularization did not significantly improve performance. Instead, it helped stabilize the models, though the overall impact was minimal. The analysis shows that median income is the most important predictor of housing prices, and has a strong positive correlation with house values. However, the model performance was limited by underfitting, especially in the linear regression models. More advanced techniques may be needed to balance model complexity and generalization.

The classification model performed reasonably well in predicting "Low" and "High" house value categories but had trouble predicting "Medium" category, suggesting possible overlap in feature space. The evaluation metrics, including accuracy, precision, and recall, were consistent, but further tuning and advanced methods, such as using more complex algorithms or refining feature selection, could enhance performance, particularly for more nuanced categories. Future improvements should focus on addressing underfitting by introducing more sophisticated algorithms or additional feature engineering to better capture the complexity of the relationships between features.

# 2. Data Exploration

## 2.1 Preview the Data

| | longitude | latitude | housing_median_age | total_rooms | total_bedrooms | population | households | median_income | median_house_value | ocean_proximity |
|---|---|---|---|---|---|---|---|---|---|---|
| **0** | -122.23 | 37.88 | 41.0 | 880.0 | 129.0 | 322.0 | 126.0 | 8.3252 | 452600.0 | NEAR BAY |
| **1** | -122.22 | 37.86 | 21.0 | 7099.0 | 1106.0 | 2401.0 | 1138.0 | 8.3014 | 358500.0 | NEAR BAY |
| **2** | -122.24 | 37.85 | 52.0 | 1467.0 | 190.0 | 496.0 | 177.0 | 7.2574 | 352100.0 | NEAR BAY |
| **3** | -122.25 | 37.85 | 52.0 | 1274.0 | 235.0 | 558.0 | 219.0 | 5.6431 | 341300.0 | NEAR BAY |
| **4** | -122.25 | 37.85 | 52.0 | 1627.0 | 280.0 | 565.0 | 259.0 | 3.8462 | 342200.0 | NEAR BAY |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **20635** | -121.09 | 39.48 | 25.0 | 1665.0 | 374.0 | 845.0 | 330.0 | 1.5603 | 78100.0 | INLAND |
| **20636** | -121.21 | 39.49 | 18.0 | 697.0 | 150.0 | 356.0 | 114.0 | 2.5568 | 77100.0 | INLAND |
| **20637** | -121.22 | 39.43 | 17.0 | 2254.0 | 485.0 | 1007.0 | 433.0 | 1.7000 | 92300.0 | INLAND |
| **20638** | -121.32 | 39.43 | 18.0 | 1860.0 | 409.0 | 741.0 | 349.0 | 1.8672 | 84700.0 | INLAND |
| **20639** | -121.24 | 39.37 | 16.0 | 2785.0 | 616.0 | 1387.0 | 530.0 | 2.3886 | 89400.0 | INLAND |

20640 rows × 10 columns

The dataset contains 20,640 rows and 10 columns, detailing various housing attributes. It includes geographical information like longitude and latitude, along with demographic and housing features such as housing median age, total rooms, total bedrooms, population, households, and median income. The target variable is median house value, while ocean proximity provides categorical data on the location's distance from the ocean.

## 2.2 Summary Statistic

The next step was to **calculate summary statistics**, including count, mean, standard deviation, minimum, 25th percentile, median (50th percentile), 75th percentile, maximum values and mode for the numerical variables. This helps to identify provide a quick overview of the numerical variable's distribution.

| | count | mean | std | min | 25% | 50% | 75% | max |
|---|---|---|---|---|---|---|---|---|
| **longitude** | 20640.0 | -119.569704 | 2.003532 | -124.3500 | -121.8000 | -118.4900 | -118.01000 | -114.3100 |
| **latitude** | 20640.0 | 35.631861 | 2.135952 | 32.5400 | 33.9300 | 34.2600 | 37.71000 | 41.9500 |
| **housing_median_age** | 20640.0 | 28.639486 | 12.585558 | 1.0000 | 18.0000 | 29.0000 | 37.00000 | 52.0000 |
| **total_rooms** | 20640.0 | 2635.763081 | 2181.615252 | 2.0000 | 1447.7500 | 2127.0000 | 3148.00000 | 39320.0000 |
| **total_bedrooms** | 20433.0 | 537.870553 | 421.385070 | 1.0000 | 296.0000 | 435.0000 | 647.00000 | 6445.0000 |
| **population** | 20640.0 | 1425.476744 | 1132.462122 | 3.0000 | 787.0000 | 1166.0000 | 1725.00000 | 35682.0000 |
| **households** | 20640.0 | 499.539680 | 382.329753 | 1.0000 | 280.0000 | 409.0000 | 605.00000 | 6082.0000 |
| **median_income** | 20640.0 | 3.870671 | 1.899822 | 0.4999 | 2.5634 | 3.5348 | 4.74325 | 15.0001 |
| **median_house_value** | 20640.0 | 206855.816909 | 115395.615874 | 14999.0000 | 119600.0000 | 179700.0000 | 264725.00000 | 500001.0000 |

**Analysis**: The output indicated that features such as *total_rooms*, *total_bedrooms*, *population*, and *households* exhibited highly skewed distributions. Additionally, the *total_bedrooms* feature contained missing values, which required further handling.

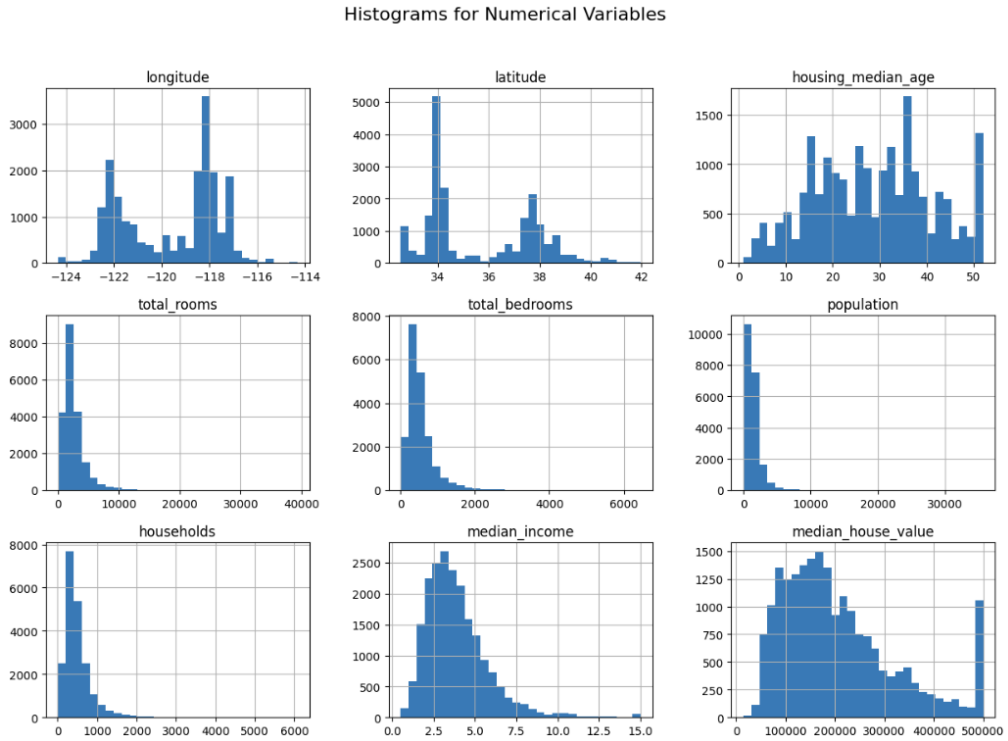
And the categorical feature ocean_proximity was summarized by computing class frequency counts.

| ocean_proximity | count |
|---|---|
| <1H OCEAN | 9136 |
| INLAND | 6551 |
| NEAR OCEAN | 2658 |
| NEAR BAY | 2290 |
| ISLAND | 5 |

**Analysis**: This distribution is imbalanced, with 5 counts of ISLAND category, which is highly underrepresented.

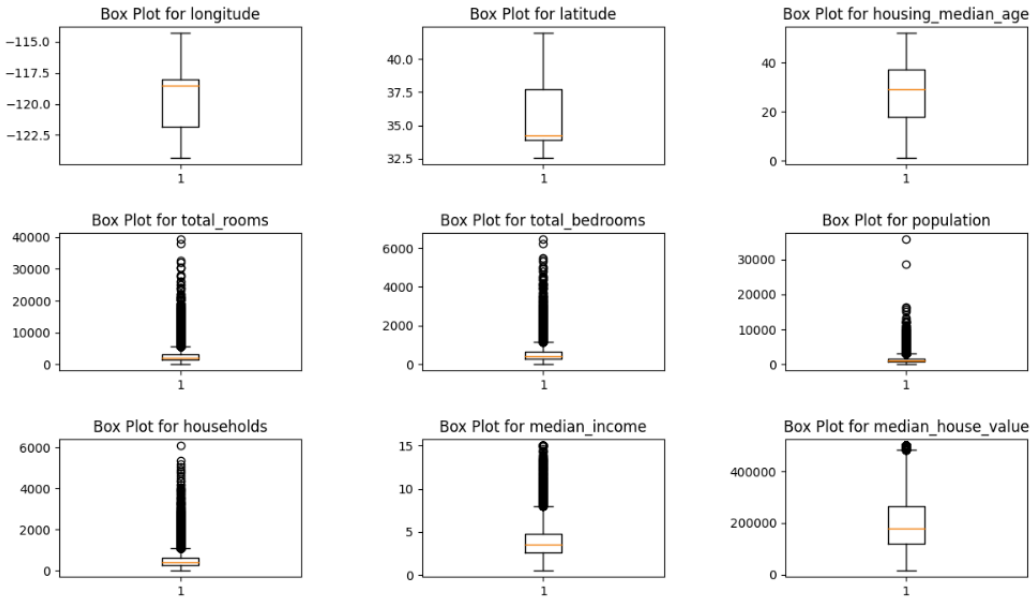
## 2.3 Data Visiualization

To gain a deeper understanding of the data, several visualizations were employed:
**Histograms**:
Histograms were plotted for all numerical variables to observe their distributions.




**Analysis**: Many of the features had long tails and skewed distributions (*total_rooms, total bedrooms, population, households*). In other words, there are many small values and few large values, which is bad for algorithms that assume normal distribution, such as linear regression. So next, we would use the Interquartile Range (IQR) methos to identify and remove outliers in data cleaning later.

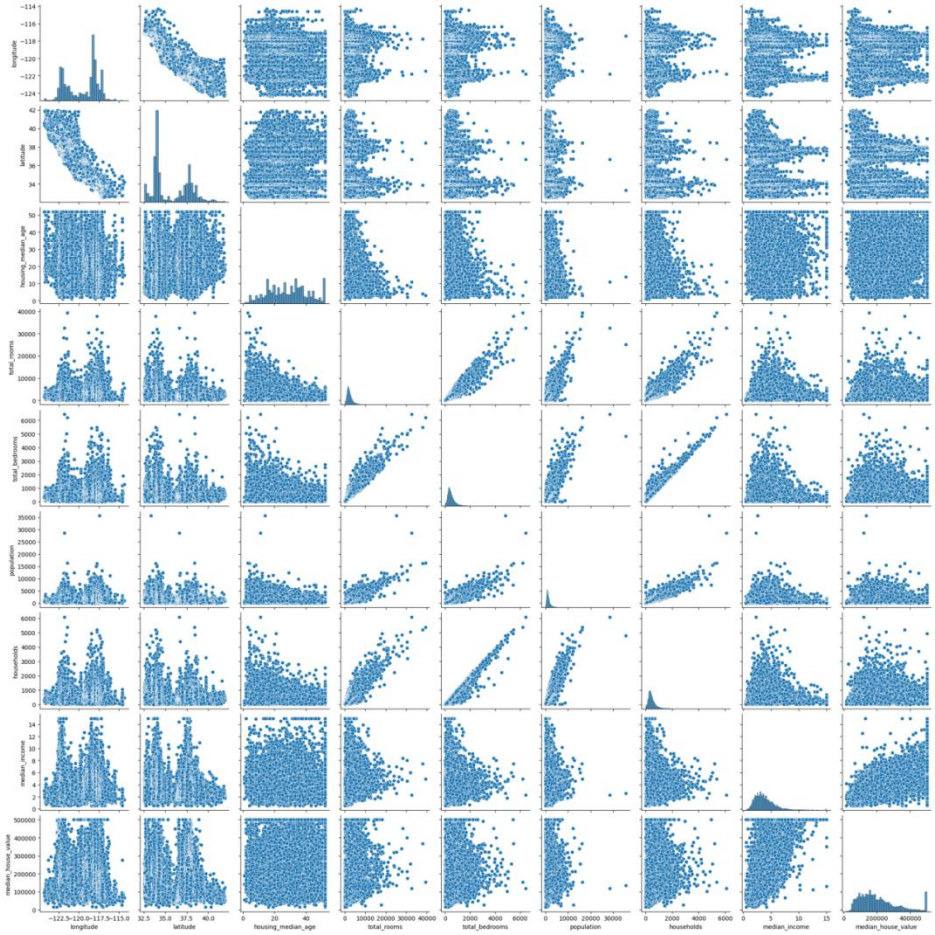
**Box Plots**:
Box plots were used to identify potential outliers in the numerical features.




**Analysis**: Many features, particularly *total_rooms, total_bedrooms, population*, and *households*, have substantial outliers. These outliers need to be addressed using techniques such as IQR filtering to improve model performance. Longitude and latitude do not have outliers, indicating that the geographical data is fairly clean.

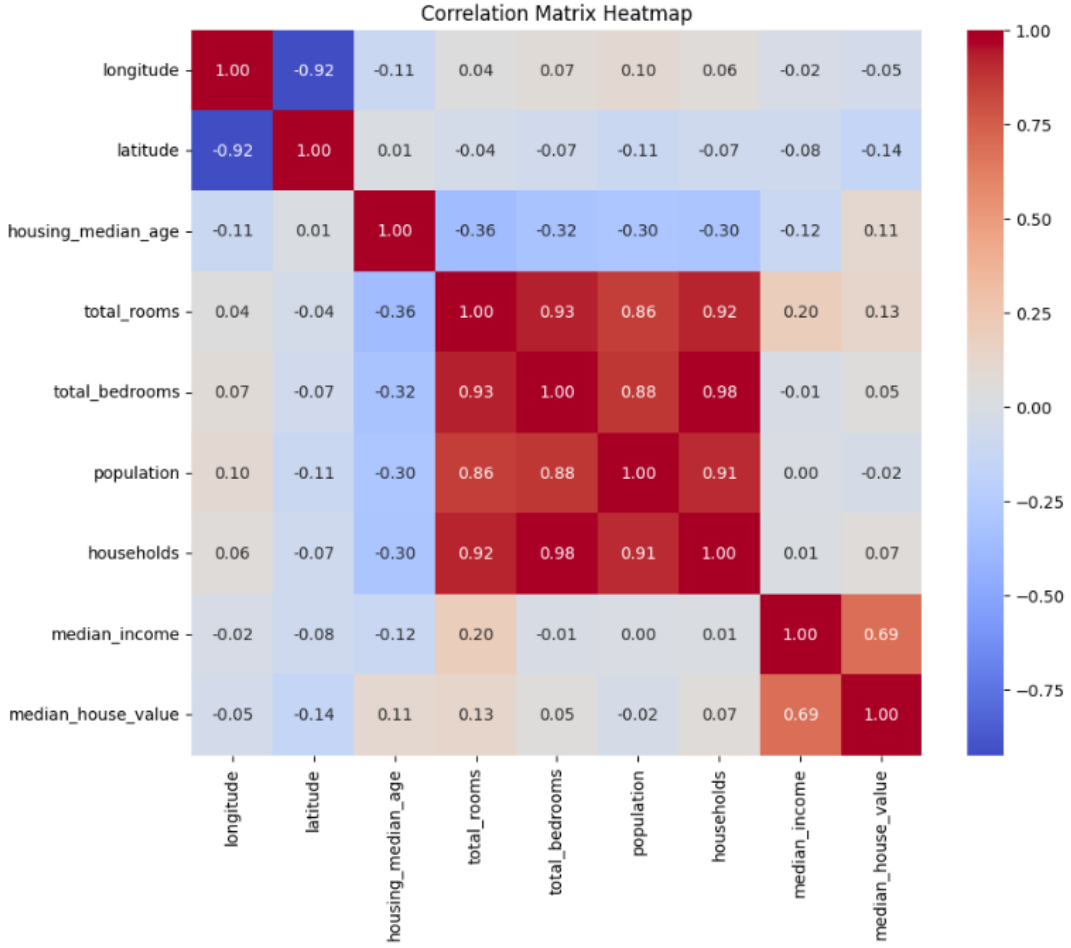
**Scatter Plots and Pairplots**:

Scatter plots and pairplots were used to explore relationships between pairs of numerical variables.



**Analysis**: There is a clear positive linear relationship between *total_rooms* and *total_bedrooms*, longitude and *latitude*, as well as between *population* and *households*. These relationships make sense since larger houses tend to have more bedrooms, and higher populations in neighborhoods correlate with more households. Similarly, *total_rooms* and *households* show a fairly strong positive correlation, which is expected as well. As the same in the histograms, many features have skewed distributions. *Median_income* has a more normalized distribution compared to other variables.

**Correlation Heatmap**:
A heatmap of the correlation matrix was generated to quantify the strength of relationships between numerical variables.




**Analysis**: It's obvious that *median_house_value* had a strong positive correlation with *median_income*, which aligns with the fact. Strong multicollinearity among *total_rooms*, *total_bedrooms*, *households* and *population* as well as *longitude* and *latitude* features suggests that **one or more of these could be removed or combined to simplify the model** without losing much information.

## 2.4 Data Processing

Handling missing values and outliers was critical to ensuring the integrity of the dataset.
**1. Filling Missing Values Using Median Value**: Since *total_bedrooms* contained missing values, we used the median of the respective feature to fill in the blank. The

median was chosen because it is less affected by outliers, maintaining the distribution's integrity without skewing it as the mean might.

**2. Feature Created and Removal to Address Multicollinearity**

In this step, we removed several features that are likely to exhibit multicollinearity, a situation where two or more features are highly correlated. Multicollinearity can cause several issues in machine learning models, especially in linear models like logistic regression or linear regression, because it makes it difficult for the model to distinguish the individual effects of these correlated features on the target variable.

As observed in the data exploration, several features—*total_rooms*, *total_bedrooms*, *population*, *households*, *longitude*, and *latitude*—exhibited multicollinearity. To address this, we created three new features:

- ***rooms_per_household***: Captures the average number of rooms per household, consolidating information from the original *total_rooms* and *households* features.
- ***bedrooms_per_population***: Represents the ratio of bedrooms to population, retaining the relationship between *total_bedrooms* and *population*.
- ***longitude_divide_latitude***: A new feature that captures the relationship between longitude and latitude, combining their geographic information into a single feature.

These new features retain the most relevant information from the original variables while reducing multicollinearity and redundancy, ultimately improving the model's stability, interpretability, and performance.

```
import numpy as np
# Create a new feature: average number of rooms per household
data['rooms_per_household'] = data['total_rooms'] / data['households']
data['bedrooms_per_poupulation'] = data['total_bedrooms'] / data['population']
data['longitude_divide_latitude'] = data['longitude'] / data['latitude']

# Removal of multicollinearity and features
data = data.drop(columns=['total_rooms', 'total_bedrooms', 'population', 'households', 'longitude', 'latitude'])
```

**3. One-Hot Encoding**: The categorical feature *ocean_proximity* was transformed into numerical form using one-hot encoding. This step converted the categorical data into a format suitable for machine learning algorithms without losing any information.

```
# one-hot encoding
data_encoded = pd.get_dummies(data, columns=['ocean_proximity'], dtype = int)
```

**4. Outlier Removal Using IQR**: Outliers were removed using the Interquartile Range (IQR) method, which is effective for handling extreme values in skewed data distributions. We define a value as an outlier based on the Interquartile Range (IQR) and the IQR is the range between third quartile (Q3) and the first quartile (Q1). A value is outlier if it is less than Q1-1.5*IQR or great than Q3+1.5IQR. The shape of the cleaned data is 18084 rows and 11 columns, meaning that we remove 2556 outliers.

```
numerical_features = data_encoded.select_dtypes(include=['float64']).columns

# Detect and remove outliers using the IQR method
def remove_outliers_iqr(df, features):
    for feature in features:
        Q1 = df[feature].quantile(0.25)
        Q3 = df[feature].quantile(0.75)
        IQR = Q3 - Q1

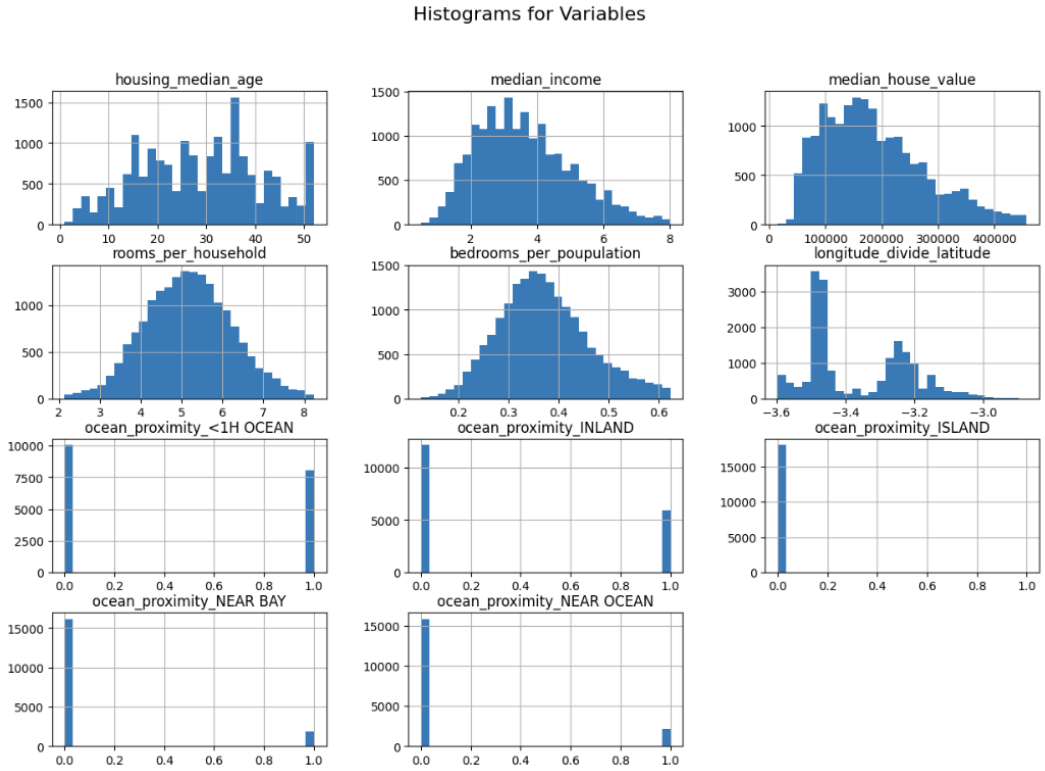
        # lower and upper bounds for outliers
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        # Remove outliers
        df = df[(df[feature] >= lower_bound) & (df[feature] <= upper_bound)]

    return df

data_cleaned = remove_outliers_iqr(data_encoded, numerical_features)
```

The histograms below show the distribution of the variables in the dataset after processing.




**Analysis**: After processing, the variables are in much better shape then before, but there are still some skewed distribution. For the categorical feature ocean_proximity, one-hot encoding has been successfully applied, though the imbalance in the ISLAND category remains noticeable, with very few instances. This imbalance may still pose challenges during modeling, especially in accurately predicting underrepresented categories.

# 3. Probability Theory Application

In this section, we apply probability theory to the *median_income* feature of the dataset, fitting normal distribution and evaluating its goodness-of-fit. We also compute the conditional probability of the target variable (*median_house_value*) based on other features (*median_income* and *housing_median_age*). Finally, we calculate the expected value and variance of a significant feature to understand its behavior and influence on feature selection and modeling.

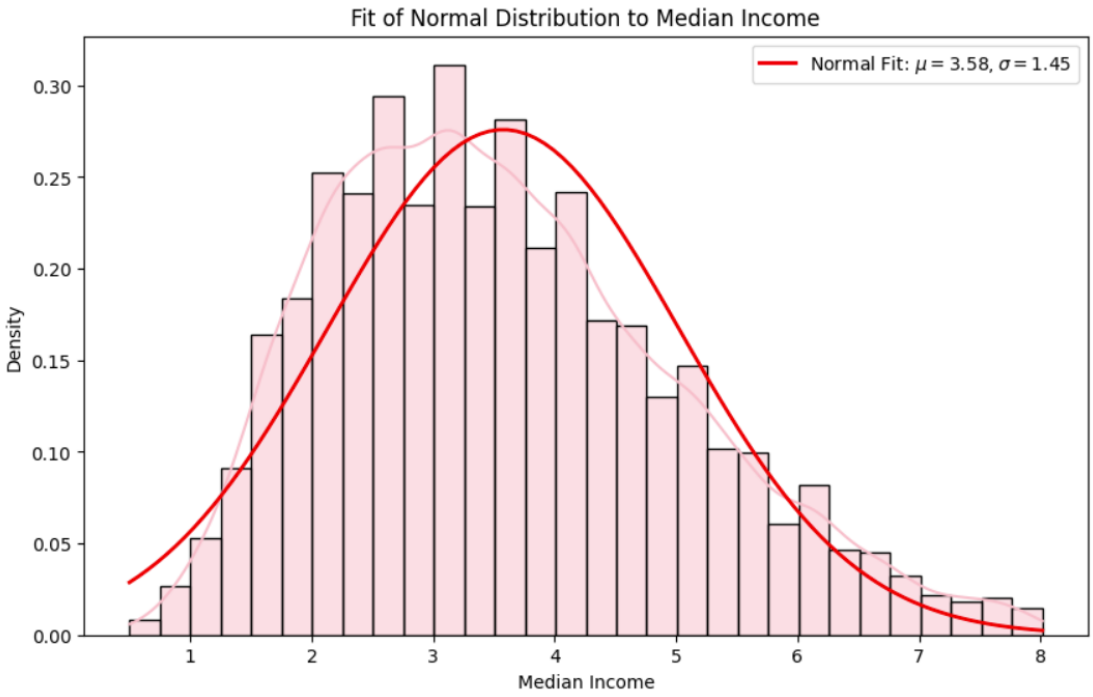
## 3.1 Normal Distribution

First, we chose *median_income* as the key feature and fit it to a normal distribution. The normal distribution provides a theoretical framework for understanding the central tendency and variability of the data. We plotted the distribution of *median_income* and overlaid the best-fitting normal distribution using the parameters for mean (μ) and standard deviation (σ) obtained from the fit.

```
# Plot the histogram and fit the normal distribution
plt.figure(figsize=(10, 6))
sns.histplot(df[key_feature], bins=30, kde=True, color='pink', stat='density')
mu, std = stats.norm.fit(df[key_feature])
x = np.linspace(df[key_feature].min(), df[key_feature].max(), 100)
p = stats.norm.pdf(x, mu, std)

# Plot the fitted normal distribution
plt.plot(x, p, 'r', linewidth=2, label=f'Normal Fit: $\mu={mu:.2f}, \sigma={std:.2f}$')
plt.title('Fit of Normal Distribution to Median Income')
plt.xlabel('Median Income')
plt.ylabel('Density')
# Add a legend to distinguish between data and fitted distribution
plt.legend(loc='upper right', fontsize=10)

plt.show()
```




**Analysis**:

The red curve represents the fitted normal distribution with a mean (μ) of 3.58 and a standard deviation (σ ) of 1.45. The pink bars show the histogram of the actual median_income values in the dataset with the overlaid pink curve.

While the normal distribution fits the central part of the data well, the tails of the distribution are not perfectly captured by the normal fit.

- Left tail: The left side (lower income values) shows a steeper drop compared to the normal distribution curve, indicating that there are fewer households with very low incomes than the normal distribution would predict.
- Right tail: The right side of the distribution (higher incomes) falls fater than the fitted curve, suggesting that there are more higher-income households than what a normal distribution would expect at the extreme right.

Although the tails on both sides are slightly biased, in general, the fitted normal distribution can effectively describe most of the features of the data, especially the data close to the mean.

**Kolmogorov-Smirnov**

To formally assess the goodness-of-fit, we performed a Kolmogorov-Smirnov (K-S) test. The K-S test compares the empirical distribution of the data with the theoretical normal distribution. The results are as follows:

- K-S Statistic: Measures the maximum difference between the empirical and theoretical cumulative distribution functions.
- P-Value: The null hypothesis used to test whether the data follows a normal distribution. If the p-value is less than the significance level ($\alpha = 0.05$), we reject the null hypothesis.

```
# K-S test
ks_statistic, p_value = stats.kstest(df[key_feature], 'norm', args=(mu, std))

# print K-S result
print(f'K-S Statistic: {ks_statistic}')
print(f'P-Value: {p_value}')

# hypothesis test using p-value
alpha = 0.05  # significance level
if p_value < alpha:
        print("Rejection of the null hypothesis: data do not follow a normal distribution")
else:
        print("Null hypothesis cannot be rejected: data follow a normal distribution")
```

```
K-S Statistic: 0.05515386237373526
P-Value: 4.099646168335164e-45
Rejection of the null hypothesis: data do not follow a normal distribution
```

**Analysis**:

- **K-S Statistic(0.0551):** This value represents the maximum distance between the empirical distribution of the median_income data and the theoretical normal distribution.
- **P-Value(4.09e-45):** The p-value is extremely small, approximately 0, much less than the significance level ($\alpha = 0.05$).
- **Null Hypothesis ($H_0$)**: The data follows a normal distribution.
- **Alternative Hypothesis ($H_1$)**: The data does not follow a normal distribution.

Since the p-value is significantly lower than 0.05, the null hypothesis is rejected. This means that, based on the K-S test, the median_income data does not follow a normal distribution.

## 3.2 Conditional Probability

Next, we analyzed the conditional probability of *median_house_value* based on *median_income* and *housing_median_age*. To facilitate this analysis, we discretized the continuous target variable *median_house_value* into three categories: "Low" "Medium" and "High" using quartiles to define the boundaries.

```
# Calculation of quartiles
q25 = df['median_house_value'].quantile(0.25)
q50 = df['median_house_value'].quantile(0.5)
q75 = df['median_house_value'].quantile(0.75)

# Setting interval
bins = [-float('inf'), q25, q75, float('inf')]
labels = ['Low', 'Medium', 'High']

# Discret median_house_value
df['house_value_category'] = pd.cut(df['median_house_value'], bins=bins, labels=labels)
```

By calculating the mean *median_house_value* for different combinations of *median_income* and *housing_median_age*, we explored how these features influence house prices.

```
# Calculate the conditional probability of median house value given certain features
# use groupby
conditional_prob = df.groupby(['median_income', 'housing_median_age'])['median_house_value'].mean().reset_index()

plt.figure(figsize=(12, 6))
sns.scatterplot(data=df, x='median_income', y='housing_median_age',
                size='median_house_value', sizes=(20, 200), alpha=0.5,
                hue='median_house_value', palette='viridis')
plt.title('Scatter Plot of Median Income vs Housing Median Age')
plt.xlabel('Median Income')
plt.ylabel('Housing Median Age')
plt.show()
```




**Analysis**: The scatter plot generated from this analysis reveals that areas with higher *median_income* tend to have higher house prices, confirming that *median_income* is a key feature for predicting house prices. On the other hand, *housing_median_age* appears to have a more subtle effect on house prices, showing a weaker relationship.

## 3.3 Expected Value and Variance

To further understand the behavior of the *median_income* feature, we computed its expected value (mean) and variance.

```
expected_value = df['median_income'].mean()
variance = df['median_income'].var()

print(f'Expected Value (Mean) of Median Income: {expected_value:.2f}')
print(f'Variance of Median Income: {variance:.2f}')
```

```
Expected Value (Mean) of Median Income: 3.58
Variance of Median Income: 2.09
```

**Analysis**:
The mean value of median income is 3.58, which gives us the central tendency of the distribution. This means that, on average, households in the dataset have a median income of 3.58.

The variance is 2.09, which measures the spread or dispersion of the data. A higher variance indicates more variability in income levels across different households in the dataset, while a lower variance would suggest incomes are closer to the mean. In this case, the variance shows moderate variability in the median income values, meaning that while most values are close to the mean, there are some incomes further away from the average.

### 3.4 Insights and Impact on Feature Selection

By applying probability theory, *median_income* was identified as a key feature because of its central tendency and moderate variability, making it a strong predictor of *median_house_value*. Conditional probability analysis further confirmed that higher *median_income* is strongly correlated with higher house prices, underscoring its significance in the model. Although *housing_median_age* showed a weaker relationship with house prices, it was still included for its potential subtle influence. While the Kolmogorov-Smirnov test showed that *median_income* does not perfectly follow a normal distribution, its alignment near the mean supported its use but may need further addresss. These findings refined feature selection and highlighted the need for potential transformations to manage skewness, ensuring more robust modeling.

## 4. Regression Modeling

The dataset was split into training and testing sets using an 80/20 ratio. This ensures that 80% of the data is used to train the model, while 20% is reserved for testing, allowing for an evaluation of the model's performance on unseen data. By using a random_state=42, the split remains consistent for reproducibility purposes.

```
X = df.drop(columns='median_house_value')    # Features
y = df['median_house_value']    # Target variable
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

### 4.1 Linear Regression

A linear regression model was developed to predict the target variable, median house value. The model was trained on the training dataset and then applied to the testing set to evaluate its predictive power.

The linear model is represented as:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots\ldots + \beta_n x_n$$

Where:

- y is the predicted median house value,
- $x_1, x_2, \ldots, x_n$ are the features,
- $\beta_0$ is the intercept, and
- $\beta_0$, $\beta_1$, $\beta_2$,..., $\beta_n$ are the coefficients for each feature.

```
# Create and fit the linear regression model
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Predict on the training and testing sets
y_train_pred = linear_model.predict(X_train)
y_test_pred = linear_model.predict(X_test)
```

Three key performance metrics were used to evaluate the linear regression model:

**Mean Squared Error (MSE)**

MSE measures the average squared difference between the actual and predicted values. A lower MSE indicates a better fit.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

**Root Mean Squared Error (RMSE)**

RMSE is the square root of the MSE, providing an interpretation in the same units as the target variable (median house value).

$$\text{RMSE} = \sqrt{\text{MSE}}$$

**R-squared ($R^2$)**

$R^2$ measures the proportion of variance in the target variable that is predictable from the features. An $R^2$ value close to 1 indicates a strong model, while a value closer to 0 suggests poor predictive ability.

$$R^2 = 1 - \frac{\sum(y - \hat{y})^2}{\sum(y - \bar{y})^2}$$

```
# Calculate performance metrics
mse_train = mean_squared_error(y_train, y_train_pred)
rmse_train = mse_train ** 0.5
r2_train = r2_score(y_train, y_train_pred)

mse_test = mean_squared_error(y_test, y_test_pred)
rmse_test = mse_test ** 0.5
r2_test = r2_score(y_test, y_test_pred)
```

The result is:

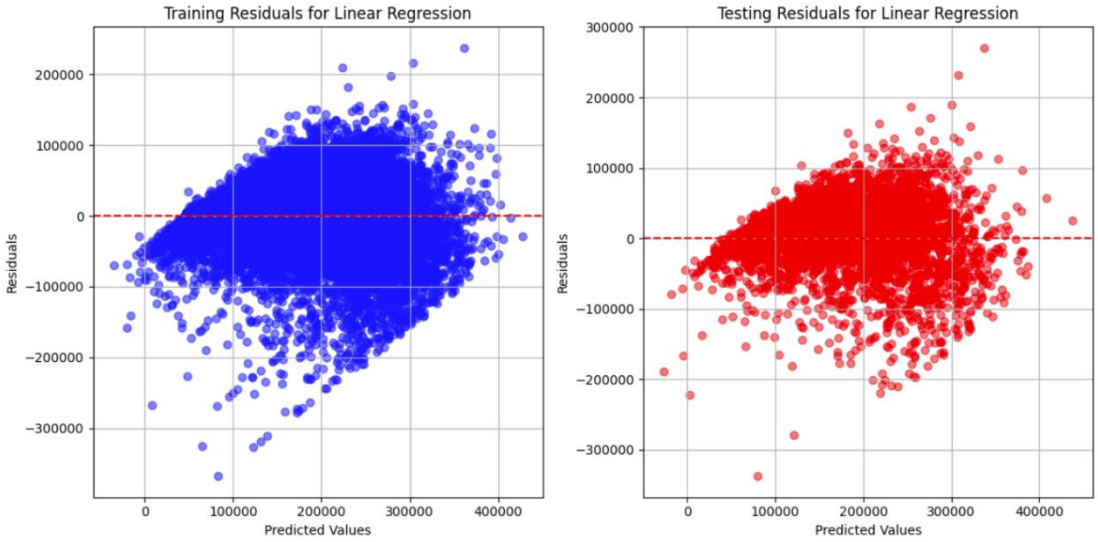
```
Training MSE: 2858034768.21, RMSE: 53460.59, R²: 0.65
Testing MSE: 2770910021.68, RMSE: 52639.43, R²: 0.65
```

**Analysis**:

- The $R^2$ values for both the training and testing sets are the same (0.65), indicating the model explains 65% of the variance in the target variable, which suggests a decent model fit but still leaves room for improvement.
- The MSE and RMSE values are close between the training and testing sets, which suggests that the model generalizes well and is not overfitting. However, compared, with the range of the *median_house_value* variable *(*approximately 14999 to 500001), an RMSE of around 52,000 suggests that the model is making errors that are significant in relation to the actual house prices.

**Residual**

Residual plots were created to evaluate the model's assumptions and potential issues:




**Analysis**:
In a well-behaved linear regression model, the residuals should be randomly scattered around zero with constant variance (i.e., the spread of residuals should remain consistent across the range of predicted values).

However, the residuals show a fan-like shape in both training and testing set. In the training residuals plot, the model performs better for lower predicted values and worse as predictions increase, meaning that the error variance is not constant across all predicted values. In the testing residuals, a similar pattern is observed, with residuals more spread out at higher predicted values. This pattern suggests the model may not be fully capturing the underlying relationships in the data.

**This kind of pattern implies that the model is not overfitting but might be underfitting**, meaning it is too simple to capture the underlying structure of the data. A more complex model (e.g., polynomial regression or adding interaction terms) might capture these relationships better.

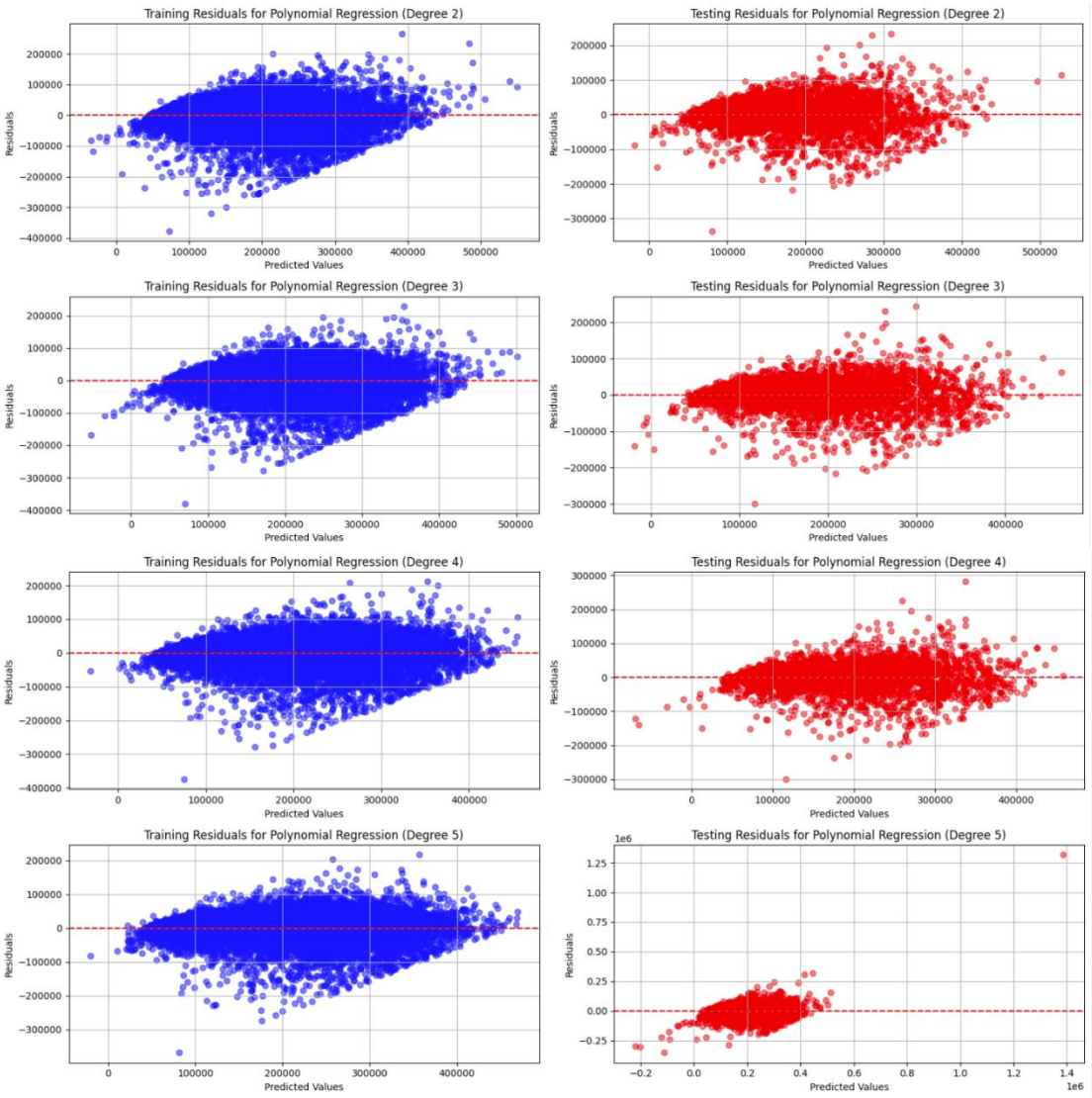
## 4.2 Polynomial Regression

Polynomial regression models of degrees 2, 3, 4, and 5 were implemented to see if introducing non-linear terms improves model performance. The goal was to observe whether increasing model complexity leads to better prediction on both training and testing data.
Polynomial regression introduces interaction and higher-order terms to the model. For example, a second-degree polynomial regression for a feature x would include terms $x^2$ along with the original feature:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_1^2 + \beta_3 x_2 + \beta_4 x_2^2 + \ldots + \beta^2 x_n^2$$

**Performance Evaluation and Residual:**

```
Polynomial Regression (Degree 2)
Training MSE: 2350331140.59, RMSE: 48480.21, R²: 0.71
Testing MSE: 2267537173.01, RMSE: 47618.66, R²: 0.72
--------------------------------------------
Polynomial Regression (Degree 3)
Training MSE: 1957858387.05, RMSE: 44247.69, R²: 0.76
Testing MSE: 1964348353.80, RMSE: 44320.97, R²: 0.75
--------------------------------------------
Polynomial Regression (Degree 4)
Training MSE: 1739434585.25, RMSE: 41706.53, R²: 0.79
Testing MSE: 1847098003.77, RMSE: 42977.88, R²: 0.77
--------------------------------------------
Polynomial Regression (Degree 5)
Training MSE: 1485870968.18, RMSE: 38547.00, R²: 0.82
Testing MSE: 2546853323.34, RMSE: 50466.36, R²: 0.68
--------------------------------------------
```




As the degree of the polynomial increases, the model becomes more complex, fitting the training data better (higher training $R^2$). But all three polynomial models show severe overfitting, where the testing MSE is extremely high, and the $R^2$ is negative. The increasing spread of residuals suggests that the model's assumptions may not hold,

particularly with respect to error variance. These models fit the training data well but performs very poorly on unseen data.

### 4.3 Model Comparison

While linear regression may not perform as well as higher-degree polynomial regression on the training data, it exhibits better performance on the test data, showing greater generalization ability. The linear regression model is more stable overall, but still underfitting.

In contrast, although higher-degree polynomial regression fits the training data exceptionally well, it suffers from overfitting, leading to poor performance on the test data and a lack of generalization.

In this case, linear regression provides a better balance between training and test performance. Unless there is a strong need to capture complex non-linear relationships, using high-degree polynomial models is not recommended.

## 5. Classification Modeling

### 5.1 Categorical Target Variable Creation

The dataset originally contains a continuous variable, *median_house_value*, which is used as the target in the classification task. To convert this continuous target variable into a categorical one, the house values were binned into three classes: Low, Medium, and High. These classes were determined using the quantile method, where the 1/3 and 2/3 quantiles of median_house_value were used as thresholds. The target values were then mapped into these discrete classes:

- Low: Values below the 1/3 quantile.
- Medium: Values between the 1/3 and 2/3 quantile.
- High: Values above the 2/3 quantile.

```
# by quantile
low, medium = df['median_house_value'].quantile([1/3, 2/3])

def categorize_value(value):
        if value <= low:
                return 'Low'
        elif value <= medium:
                return 'Medium'
        else:
                return 'High'

# Create the categorical target variable
df['house_value_category'] = df['median_house_value'].apply(categorize_value)

# Encode the target variable
category_mapping = {'Low': 0, 'Medium': 1, 'High': 2}
df['house_value_category'] = df['house_value_category'].map(category_mapping)
df = df.drop(columns=['median_house_value'])
df.to_csv("/content/drive/MyDrive/A1_Classification/data/cleaned_data_classification.csv", index = False)
```

### 5.2 Logistic Regression Model

A logistic regression model was implemented to predict the newly created categorical variable. The model was trained using the train_test_split method, where the dataset was divided into 80% training data and 20% testing data. The features were also

standardized using StandardScaler to ensure that all variables are on the same scale, which helps to improve model performance.

```
# Scale the features
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)
```

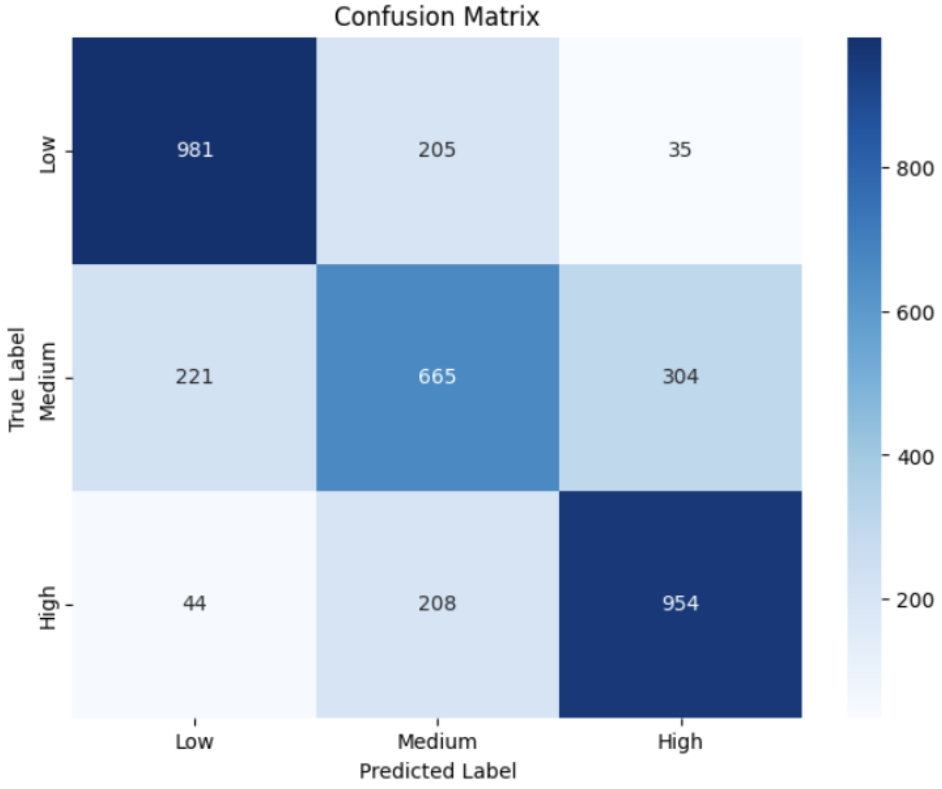
## 5.3 Model Evaluation

The model's performance was evaluated using several common classification metrics:

```
# Evaluation Metrics
conf_matrix = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
```

**Confusion Matrix**

The confusion matrix below provides an overview of how well the model predicted each class. Below shows a heatmap, with actual categories on the y-axis and predicted categories on the x-axis. The diagonal cells represent the correct predictions, while off-diagonal cells represent misclassifications.



- Diagonal Cells (Correct Predictions):
  - Low Class: 981 instances were correctly classified as "Low".
  - Medium Class: 665 instances were correctly classified as "Medium".
  - High Class: 954 instances were correctly classified as "High".
- Off-Diagonal Cells (Misclassifications):
  - Low → Medium: 205 instances of "Low" were misclassified as "Medium".
  - Low → High: 35 instances of "Low" were misclassified as "High".
  - Medium → Low: 221 instances of "Medium" were misclassified as "Low".
  - Medium → High: 304 instances of "Medium" were misclassified as "High".
  - High → Low: 44 instances of "High" were misclassified as "Low".
  - High → Medium: 208 instances of "High" were misclassified as "Medium".

**Analysis**:
The model performs well in classifying Low and High categories, as shown by the large values on the diagonal (981 for "Low" and 954 for "High"). The Medium category has the most misclassifications and it is more likely to be confused between the Low and Medium categories and between Medium and High, indicating the model has some difficulty distinguishing between adjacent classes.

The confusion matrix indicates that the logistic regression model has a strong ability to distinguish Low and High house values but struggles more with the Medium class. This kind of misclassification pattern is common when there is overlap in the feature space between adjacent categories. It might be better using more complex models (like random forests or support vector machines) or adjust feature selection to improve the classification performance, particularly for the Medium class.

**Accuracy, Precision, Recall, and F1-Score**
- Accuracy measures the overall correctness of the model, defined as the ratio of correct predictions (both true positives and true negatives) to the total number of predictions.
- Precision is the ratio of correctly predicted positive instances (true positives) to the total predicted positives (true positives + false positives).
- Recall is the ratio of correctly predicted positives to the total actual positives (true positives + false negatives).
- The F1-score is the harmonic mean of precision and recall, providing a single measure that balances both concerns.

Result:

| Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|
| 71.88% | 0.71 | 0.72 | 0.72 |

An accuracy of 71.88% means that the model correctly classified house values into the "Low", "Medium", and "High" categories 71.88% of the time.
A precision score of 0.71 implies that, when the model predicts a class, 71% of those predictions are correct.
A recall score of 0.72 indicates that the model correctly identifies 72% of the actual instances for each class.
An F1-score of 0.72 indicates that the model performs equally well in terms of precision and recall. It's a balanced metric that captures the trade-off between the two.

All the evaluation metrics—accuracy, precision, recall, and F1-score—are around 0.72, indicating that the model has consistent performance across these metrics.
Given the scores, the model is effective at predicting house value categories, though the Medium category likely still poses some challenges based on previous confusion matrix analysis.

**ROC Curve and AUC**

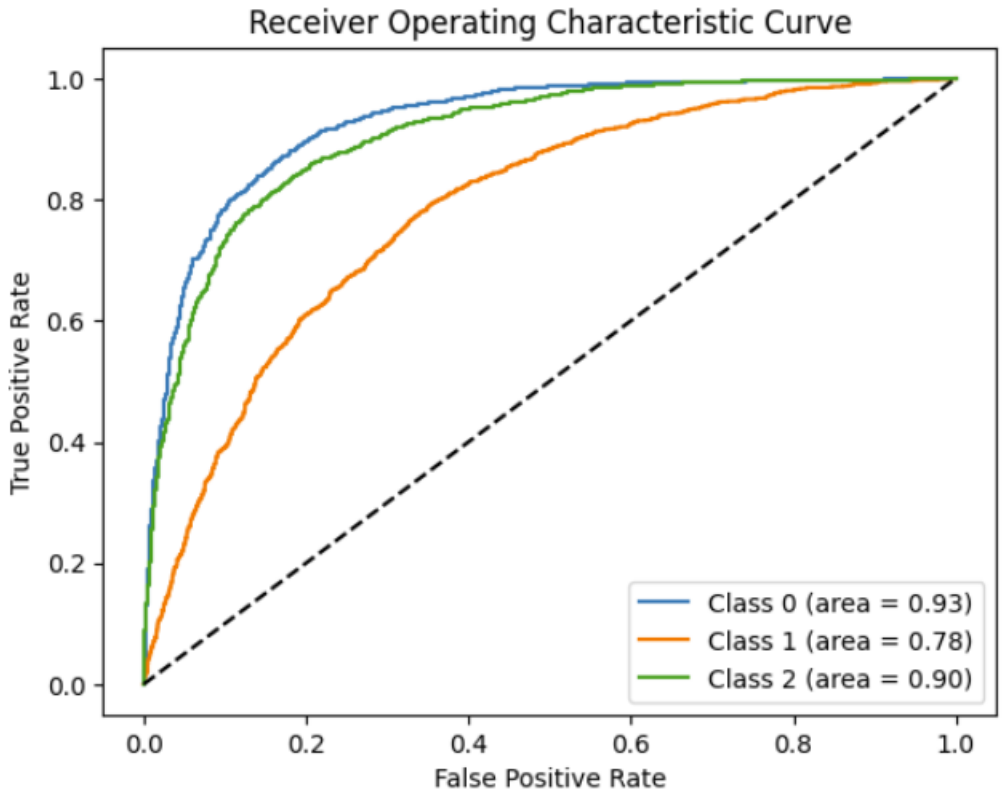
To assess the model's ability to discriminate between the three classes, a Receiver Operating Characteristic (ROC) curve was plotted for each class. The Area Under the Curve (AUC) was calculated to quantify the model's classification performance.

```
y_prob = model.predict_proba(X_test)
fpr = {}
tpr = {}
roc_auc = {}

for i in range(3):
    fpr[i], tpr[i], _ = roc_curve(y_test == i, y_prob[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

plt.figure()
for i in range(3):
    plt.plot(fpr[i], tpr[i], label='Class {} (area = {:.2f})'.format(i, roc_auc[i]))

plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic Curve')
plt.legend(loc='lower right')
plt.show()
```



**Analysis:**

- Class 0 (AUC = 0.93): The ROC curve for Class 0 shows strong performance, with an AUC of 0.93. This indicates the model effectively identifies Class 0 ("Low" house values) with a high true positive rate and few false positives.
- Class 1 (AUC = 0.78): Class 1 has a lower AUC of 0.78, suggesting the model struggles more to distinguish Class 1 ("Medium" house values) from the others. This is likely due to the overlap between Class 1 and the adjacent classes, making the boundaries less clear.
- Class 2 (AUC = 0.90): With an AUC of 0.90, the model performs almost as well for Class 2 ("High" house values) as for Class 0, indicating good discrimination ability and a low false positive rate for this class.

This ROC curve analysis is consistent with what was previously observed from the confusion matrix and the evaluation metrics where Class 1 had higher misclassification rate. This may require further regularization.

**Cross-Validation**

The k-fold cross-validation was performed to evaluate the model's robustness by splitting the dataset into 5 different folds. Each fold is used as a test set while the remaining folds are used for training, and this process is repeated 5 times.

```
import matplotlib.pyplot as plt
import numpy as np

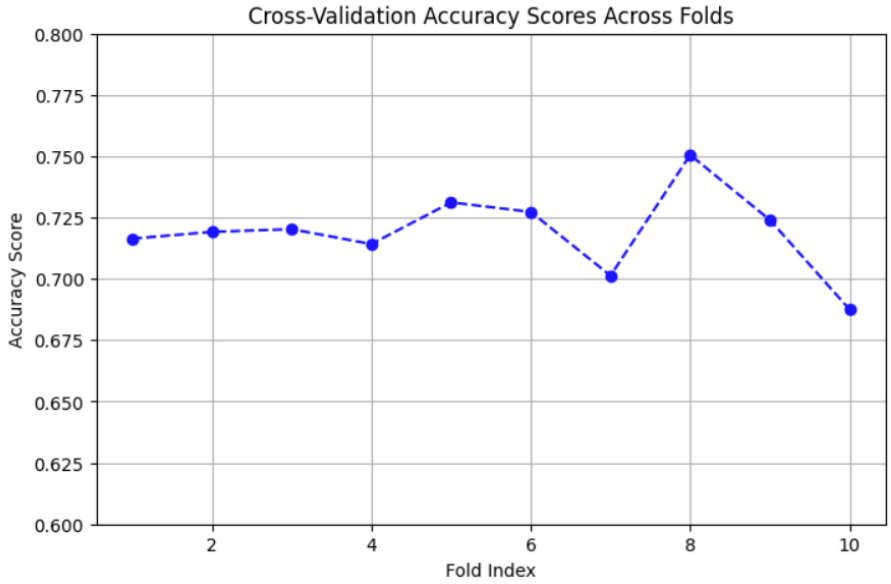
# Perform cross-validation with k-fold
kfold = KFold(n_splits=10, random_state=42, shuffle=True)
cross_val_scores = cross_val_score(model, X, y, cv=kfold, scoring='accuracy')

# Print cross-validation scores for each fold
print("Cross-validation accuracy scores for each fold:", cross_val_scores)

# Calculate and print mean, variance, and standard deviation
mean_cv_score = np.mean(cross_val_scores)
variance_cv_score = np.var(cross_val_scores)
std_cv_score = np.std(cross_val_scores)

print(f"Mean cross-validation accuracy: {mean_cv_score:.2f}%")
print(f"Variance of cross-validation accuracy: {variance_cv_score:.4f}")
print(f"Standard deviation of cross-validation accuracy: {std_cv_score:.4f}")
```

```
Cross-validation accuracy scores for each fold: [0.71641791 0.71918187 0.72028745 0.71420674 0.73119469 0.72732301
 0.70132743 0.7505531  0.72400442 0.6875    ]
Mean cross-validation accuracy: 0.72%
Variance of cross-validation accuracy: 0.0003
Standard deviation of cross-validation accuracy: 0.0160
```



**Analysis**:
The cross-validation accuracy scores for each fold show some variation, but are generally consistent, ranging from 0.6875 to 0.7505. The mean cross-validation accuracy is 0.72, and the variance and standard deviation are very low (0.0003 and 0.0160, respectively), indicating a stable model with little fluctuation across the folds.

From the plot, we observe that the model's accuracy remains relatively stable between folds, but fold 8 exhibits a slight increase in accuracy, while fold 10 experiences a slight decrease. Despite these fluctuations, the differences between the folds are small, suggesting that the model generalizes well across different data subsets.

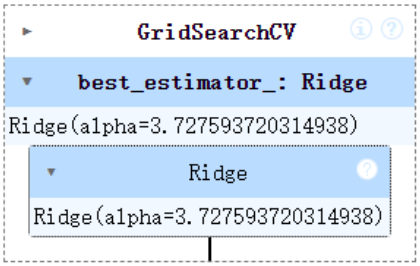
# 6. Regularization Techniques

Regularization techniques such as L1 (Lasso) and L2 (Ridge) are essential in improving the generalization of models, especially when overfitting occurs. This report explores the effects of regularization on both linear regression and logistic regression models, using cross-validation to optimize regularization parameters (alpha for regression and C for logistic regression). The focus will be on the performance metrics such as RMSE and the impact on coefficients.

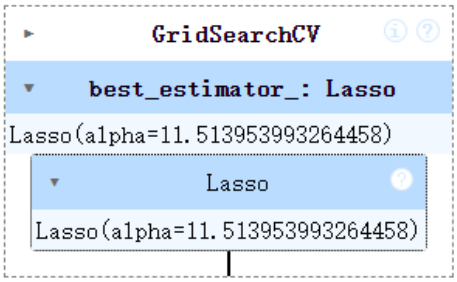
## 6.1 Linear Regression Task

For the regression task, we started with a non-regularized linear regression model and compared it with Ridge regression (L2 regularization) and Lasso regression (L1 regularization). Using GridSearchCV, we tuned the alpha parameter for both Ridge and Lasso to find the optimal regularization strength.

```
# Ridge Regression (L2 Regularization)
ridge = Ridge()
ridge_params = {'alpha': np.logspace(-4, 4, 50)} # generates 50 values for alpha
ridge_grid = GridSearchCV(ridge, ridge_params, cv=5, scoring='neg_mean_squared_error')
ridge_grid.fit(X_train_scaled, y_train)
```




```
# Lasso Regression (L1 Regularization)
lasso = Lasso()
lasso_params = {'alpha': np.logspace(-4, 4, 50)}
# Using negative MSE here is because GridSearchCV maximizes the score, so minimizir
lasso_grid = GridSearchCV(lasso, lasso_params, cv=5, scoring='neg_mean_squared_error')
lasso_grid.fit(X_train_scaled, y_train)
```



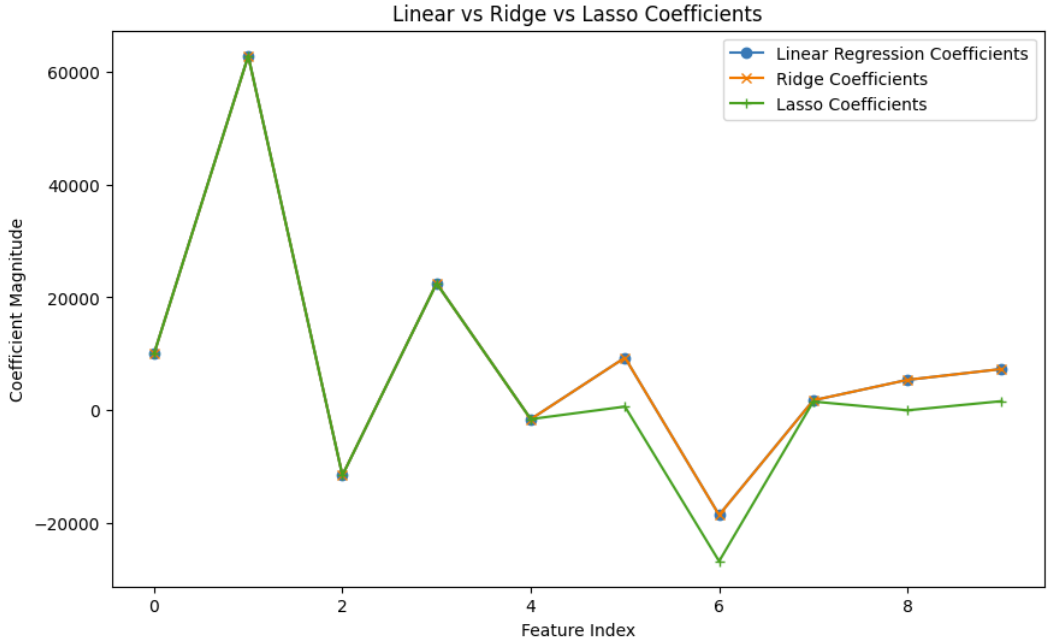

The Root Mean Squared Error (RMSE) for the regression models was computed, and the results were as follows:

| Model | RMSE |
|---|---|
| Non-Regularized Linear Regression | 54290.39 |
| Ridge Regression (L2) | 54289.18 |
| Lasso Regression (L1) | 54289.65 |

Analysis:

- Non-Regularized Linear Regression had the highest RMSE, but only marginally higher than the regularized models. The non-regularized model's coefficients tend to take larger magnitudes, as observed in the coefficient plot.
- Ridge Regression (L2 regularization) resulted in slightly lower RMSE compared to the non-regularized model. Ridge shrinks the coefficients but does not zero them out. The reduction in coefficient magnitude was evident but not drastic.
- Lasso Regression (L1 regularization) also yielded similar RMSE, indicating that regularization did not significantly impact the model's predictive performance. However, Lasso has the potential to zero out some coefficients (though it did not happen significantly in this instance).

**Comparasion:**




Linear Regression (No Regularization): Coefficients take larger magnitudes due to the absence of any penalty.
Ridge Regression (L2): Coefficients are slightly smaller compared to linear regression, indicating regularization's impact.

Lasso Regression (L1): Similar to Ridge but with potential for some coefficients to be reduced to zero.
These dataset does not suffer from multicollinearity or overfitting, the effects of regularization might not be significant, which is why the coefficients look very similar across all three models.

## 6.2 Classification Task

For the classification task, we implemented logistic regression using L2 regularization (Ridge), which penalizes large coefficients to avoid overfitting. We compared the performance of the regularized model against a non-regularized logistic regression model (by setting C=1e10 to approximate no regularization).
The C parameter in LogisticRegression controls the strength of the regularization:

- A smaller C value indicates stronger regularization, shrinking the coefficients more aggressively.
- A larger C value (like C=1e10) approximates no regularization, allowing the coefficients to grow without penalty.

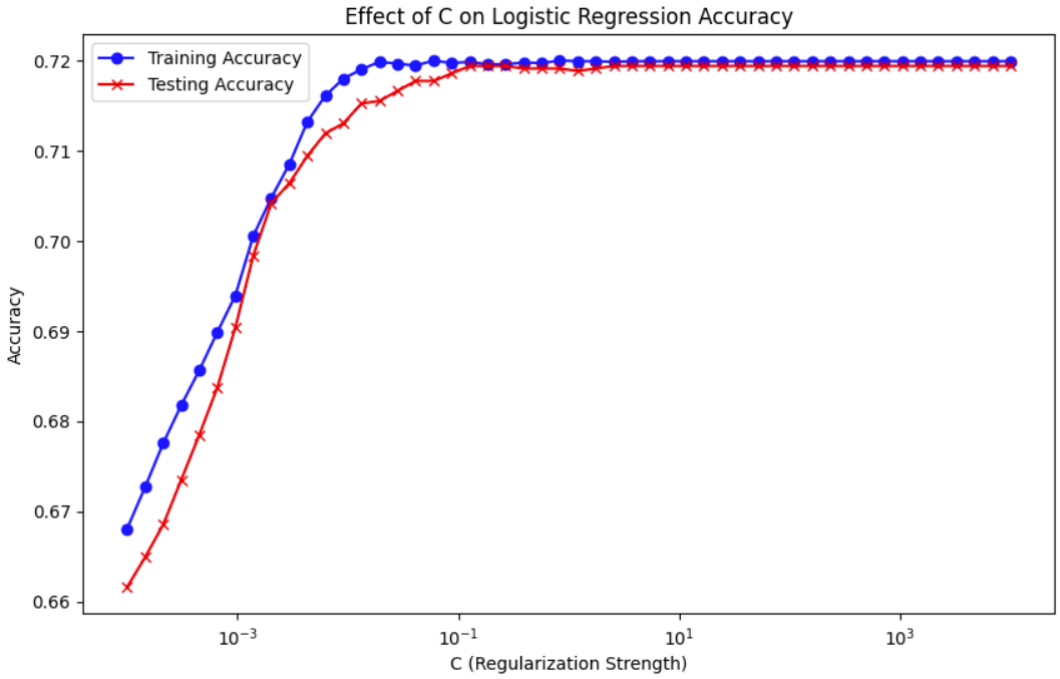
Below is to loop over each C value and shows the effect of the regularization strength parameter C on the training and testing accuracy of the logistic regression model.

```
# Loop over each C value, fit logistic regression, and track the accuracy
for C in C_values:
        logreg = LogisticRegression(C=C, max_iter=10000)
        logreg.fit(X_train_scaled, y_train)

        # Training and testing accuracy
        train_acc = accuracy_score(y_train, logreg.predict(X_train_scaled))
        test_acc = accuracy_score(y_test, logreg.predict(X_test_scaled))

        accuracy_train.append(train_acc)
        accuracy_test.append(test_acc)
```

```
# Plot the effect of varying C values on training and testing accuracy
plt.figure(figsize=(10, 6))
plt.plot(C_values, accuracy_train, label='Training Accuracy', marker='o', color='blue')
plt.plot(C_values, accuracy_test, label='Testing Accuracy', marker='x', color='red')
plt.xscale('log')
plt.xlabel('C (Regularization Strength)')
plt.ylabel('Accuracy')
plt.title('Effect of C on Logistic Regression Accuracy')
plt.legend()
plt.show()
```




**Analysis**: Both training and testing accuracy improve as C increases, but the improvement plateaus after $C \approx 10^{-1}$, with no significant gains beyond this point. This shows that overfitting is not a significant concern at higher C values.

Then we used GridSearchCV to find the optimal C value, tuning it across a wide range of values to achieve the best classification accuracy.

```
# Logistic Regression with regularization (L2 regularization)
logreg = LogisticRegression(max_iter=10000)
logreg_params = {'C': np.logspace(-4, 4, 50)}    # Varying regularization strength
logreg_grid = GridSearchCV(logreg, logreg_params, cv=5, scoring='accuracy')
logreg_grid.fit(X_train_scaled, y_train)

# Predictions
y_pred_reg = logreg_grid.best_estimator_.predict(X_test_scaled)

# Output the best regularization parameter (C)
print("Best C for regularized logistic regression:", logreg_grid.best_params_)
```

```
Best C for regularized logistic regression: {'C': 0.18420699693267145}
```

The best parameter for regularized logistic regression is approximately **0.1842**.

**Evaluation of Model:**

```
Classification Report (No Regularization):
              precision    recall  f1-score   support

           0       0.79      0.80      0.80      1221
           1       0.62      0.56      0.59      1190
           2       0.74      0.79      0.76      1206

    accuracy                           0.72      3617
   macro avg       0.71      0.72      0.72      3617
weighted avg       0.72      0.72      0.72      3617

Classification Report (With Regularization):
              precision    recall  f1-score   support

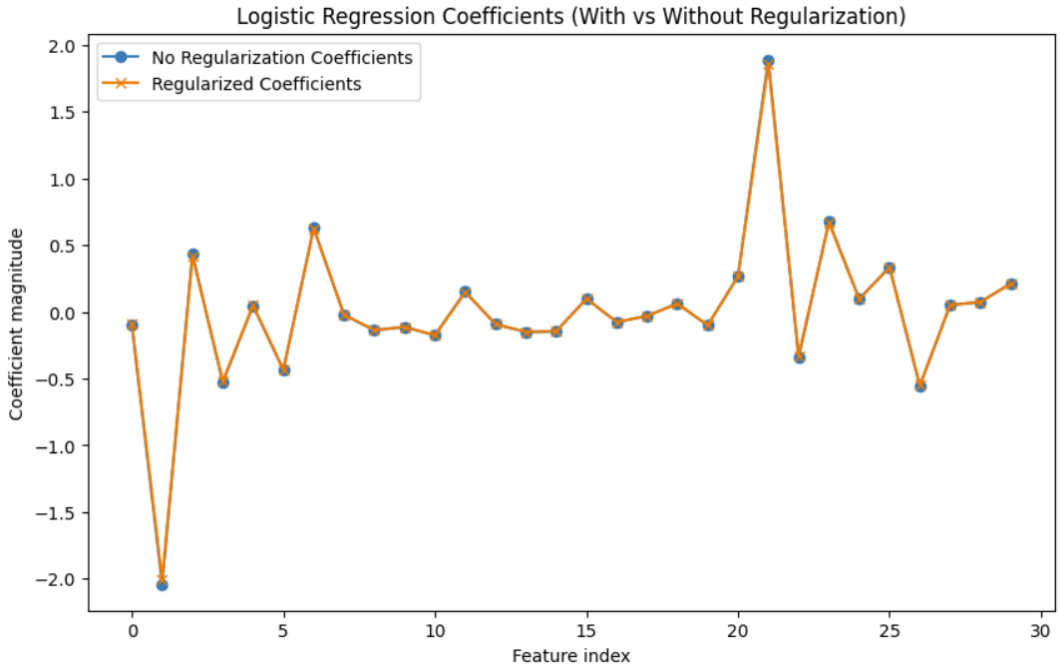
           0       0.79      0.80      0.80      1221
           1       0.62      0.56      0.59      1190
           2       0.74      0.79      0.76      1206

    accuracy                           0.72      3617
   macro avg       0.71      0.72      0.72      3617
weighted avg       0.72      0.72      0.72      3617
```

**Analysis:**

The classification report indicates that L2 regularization had minimal effect on the overall performance, with accuracy remaining steady at 72% for both regularized and non-regularized models. Precision, recall, and F1-scores were largely consistent across all classes, though class 2 (*high house values*) saw a slight improvement in F1-score with regularization. While regularization helped balance performance for class 2, it did not significantly enhance the classification of class 1 (*medium house values*), which continued to have lower metrics. Overall, the impact of regularization was minor, suggesting the model was not heavily overfitting.

**Coefficient Behavior:**




```
Best C for regularized logistic regression: {'C': 0.18420699693267145}
```

The blue and orange lines are very close to each other, almost the same trajectory, indicating regularization did not drastically affect the model's performance. This observation implies that the model was not overfitting to the data even without regularization, and regularization mainly served as a precaution to stabilize the coefficients.

# 7. Conclusion

This report analyzed the California Housing Prices dataset to predict house prices and classify them into price ranges using machine learning models. Median income was identified as the most important predictor of house prices, showing a strong positive correlation with the target variable. While we addressed multicollinearity by creating new features, the feature engineering approach is quite basic. In this case, using the method that one feature divide another to reduce multicollinearity, which may have oversimplified the relationships between variables.

The linear regression model showed signs of underfitting, meaning it didn't capture the more complex patterns in the data. On the other hand, polynomial regression overfit the training data, resulting in poor performance on test data. In classification tasks, logistic regression worked well for Low and High price categories but struggled with the Medium category, likely due to overlap between features. In the future study, we should find ways to solve this problem. Regularization techniques like Ridge and Lasso were applied to improve stability, but their effect was minimal because underfitting remained the primary issue.

Overall, while the analysis provided valuable insights, the feature engineering may not have fully addressed the complexities in the data. Future work should focus on improving feature engineering methods to better capture relationships between variables. Additionally, using more advanced algorithms and refining feature selection could help reduce underfitting and improve classification accuracy, especially for more complex categories.